

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Case Study

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA0509

COURSE OUTCOME COVERED

CO3: *Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Annexure A Case Study

Code of Conduct:

I N.Akshaya(192425129) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q. No	Understanding of Case (25%)	Application of Concepts (25%)	Analysis (20%)	Solution Design (20%)	Clarity (10%)	Total Score (100%)
1						
2						
3						
4						
5						
Total Marks (Out of 100)					Total Marks (Out of 50)	

Annexure A

Case Study

Read the given scenario carefully and analyze the problem using appropriate computer vision techniques. Justify your answers with relevant reasoning and comparisons wherever necessary.

1. Hough Transform for Shape Detection

A system uses Hough Transform to detect circular shapes but fails in complex backgrounds.

Analyze the limitations and explain how preprocessing or parameter tuning can improve detection.

Answer:

Case Understanding

The system uses the **Hough Transform to detect circular shapes**, but its performance decreases when the image contains a complex background. The main problem is that edges belonging to the background may also generate votes in the Hough accumulator, producing false circles. The case specifically asks for analysis of limitations and improvements through preprocessing and parameter tuning.

Analysis

The Circular Hough Transform detects circles by searching for possible **center coordinates and radius values**. In a complex image, many irrelevant edges can be detected. These edges increase the number of possible circle candidates.

Major limitations are:

- Background edges can create **false circle detections**.
- Noise produces unnecessary edge points.
- Incorrect radius ranges increase computational cost.
- A low detection threshold may produce too many false positives.
- A high threshold may miss actual circles.

Application of Concepts

The following preprocessing steps can improve detection:

1. Convert the image to **grayscale**.
2. Apply **Gaussian or median filtering** to reduce noise.
3. Use **Canny edge detection** to obtain meaningful edges.
4. Apply thresholding or morphological operations to remove unwanted background regions.
5. Restrict the expected **radius range** of circles.
6. Tune the Hough accumulator threshold according to the image.

Solution Design

Improved pipeline:

Input Image → Grayscale → Noise Reduction → Canny Edge Detection → Hough Circle Transform → Parameter Tuning → Detected Circles

For example, if the expected objects have radii between 20 and 60 pixels, setting the Hough Transform to search only within this range reduces unnecessary computations and false detections.

Conclusion

The Hough Transform is effective for circular shape detection, but its performance depends strongly on image quality and parameter selection. **Noise reduction, edge preprocessing, restricted radius ranges and proper accumulator thresholds** can significantly improve detection accuracy.

2. PCA in Face Recognition

A face recognition system processes high-dimensional image data, resulting in slow performance.

Analyze how PCA helps in reducing dimensionality while preserving important features.

Answer:

Case Understanding

The face recognition system processes **high-dimensional image data**, which causes slow processing. The case requires analyzing how **Principal Component Analysis (PCA)** reduces dimensionality while retaining important information.

Analysis

A face image may contain thousands of pixels. If an image has a size of 100×100 pixels, it contains:

$$100 \times 100 = 10,000 \text{ features}$$

Processing thousands of features for every face increases:

- Computational time
- Memory requirements
- Complexity of recognition
- Possibility of redundant information

Many pixel features are correlated with each other. PCA transforms the original features into a smaller set of **principal components**.

Application of PCA

The PCA process is:

1. Convert face images into numerical feature vectors.
2. Normalize or center the data.
3. Calculate the covariance matrix.
4. Calculate eigenvalues and eigenvectors.
5. Sort the principal components according to their importance.
6. Select the top **K components**.
7. Project the original face images into this reduced feature space.
8. Perform face recognition using the reduced features.

The important components capture major variations such as facial structure, shape and intensity patterns.

Example

Suppose a face image contains **10,000 original features**.

PCA may reduce these to **100 principal components** while retaining most of the useful information.

Therefore:

10,000 features → PCA → 100 features → Face Recognition

This reduces the computational workload considerably.

Solution Design

Proposed pipeline:

Face Image → Preprocessing → Feature Vector → PCA → Reduced Feature Space →
Classifier → Recognized Face

PCA should not simply reduce the number of features randomly. The components should be selected based on their explained variance so that important facial information is retained.

Conclusion

PCA is useful for face recognition because it reduces the dimensionality of high-dimensional image data while preserving the most significant variations. This makes recognition **faster, less memory-intensive and computationally efficient**.

3. Edge Detection in Low Contrast Images

An image has low contrast, causing edge detection algorithms to fail. Analyze the problem and explain how preprocessing enhances edge detection results.

Answer:

Case Understanding

The given image has **low contrast**, causing edge detection algorithms to fail. The problem requires analyzing why edges are weak and explaining how preprocessing can improve the result.

Analysis

Edge detection algorithms identify locations where there is a significant change in intensity.

In a low-contrast image, neighboring pixels may have very similar intensity values. Therefore, the intensity gradient is weak and the edge detector may not recognize important boundaries.

For example:

Object intensity = 125

Background intensity = 130

The difference is only:

$$130 - 125 = 5$$

Such a small difference can make the edge difficult to detect.

Application of Concepts

The image can be enhanced before edge detection using:

1. Contrast Stretching

Expands the intensity range and makes differences between objects and backgrounds more visible.

2. Histogram Equalization

Redistributes intensity values to improve overall contrast.

3. CLAHE

Contrast Limited Adaptive Histogram Equalization improves local contrast and is particularly useful when different regions have different illumination.

4. Noise Reduction

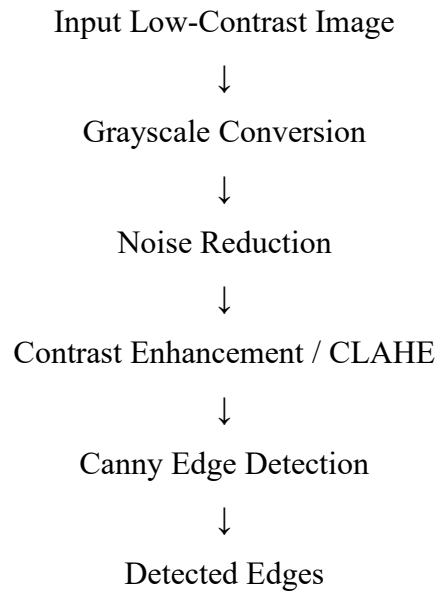
Gaussian filtering can reduce noise before applying the edge detector.

5. Canny Edge Detection

After enhancement, Canny can be applied using suitable low and high thresholds.

Solution Design

Improved pipeline:



The Canny thresholds should also be adjusted according to the intensity distribution of the enhanced image.

Comparison

Without Preprocessing	With Preprocessing
Weak edges	Stronger edges
Important boundaries may disappear	More boundaries detected
Poor edge continuity	Better edge continuity
More missed features	Better feature extraction

Conclusion

Low contrast reduces the intensity differences required for reliable edge detection. **Contrast enhancement followed by noise reduction and appropriate Canny thresholding** can significantly improve edge detection and provide better image features.

4. Corner Detection in Textured Regions

A system applies corner detection in highly textured regions, resulting in excessive keypoints.

Analyze the issue and suggest methods to improve feature selection.

Answer:

Case Understanding

The system applies corner detection to a highly textured region and obtains **excessive keypoints**. The problem is that many small intensity variations in textures may be interpreted as corners. The case asks for methods to improve feature selection.

Analysis

Corner detectors such as **Harris Corner Detection** identify points where image intensity changes significantly in multiple directions.

In a highly textured region, there may be many:

- Small edges
- Repeated patterns
- Texture variations
- High-frequency intensity changes

As a result, the detector generates too many keypoints.

This creates several problems:

- Increased computational cost
- Redundant features
- Difficult feature matching
- Increased memory usage
- Less efficient image representation

Application of Concepts

The number of useful keypoints can be controlled using:

1. **Increase the corner response threshold**
Weak corners can be removed.
2. **Non-Maximum Suppression**
Retain only the strongest keypoint within a neighborhood.

3. **Minimum Distance Constraint**

Prevent keypoints from being concentrated in the same small region.

4. **Region of Interest (ROI)**

Detect features only in relevant regions.

5. **Feature Ranking**

Rank keypoints according to their corner response and retain the strongest ones.

6. **Multi-scale detection**

Detect features at appropriate scales to reduce sensitivity to small texture variations.

Solution Design

Improved pipeline:

Input Image → Grayscale → Noise Reduction → Corner Detection → Response Thresholding
→ Non-Maximum Suppression → Strong Keypoints → Feature Description

For example, instead of keeping every detected corner, the system can retain only the **top N strongest keypoints** while maintaining a minimum distance between them.

Conclusion

Excessive keypoints in textured regions are mainly caused by repeated high-frequency intensity variations. **Thresholding, non-maximum suppression, minimum-distance filtering and keypoint ranking** can produce a smaller and more meaningful set of features.

5. Feature Matching with Noise

Feature matching between two images fails due to noise in one image. Analyze the impact of noise and explain how preprocessing improves matching accuracy.

Answer:

Case Understanding

Feature matching between two images fails because one image contains noise. Noise introduces unwanted intensity variations and can affect both feature detection and feature description. The case requires analysis of the effect of noise and preprocessing methods to improve matching accuracy.

Analysis

Feature matching generally involves:

Feature Detection → Feature Description → Feature Matching

Noise can affect all three stages.

For example, random noise may create artificial edges and corners. This can result in:

- False keypoints
- Incorrect descriptors
- Fewer correct matches
- Increased false matches
- Reduced matching accuracy

If one image is clean and the other is noisy, their corresponding feature descriptors may become significantly different even though they represent the same physical object.

Application of Concepts

Before feature extraction, noise should be reduced.

1. Gaussian Filtering

Useful for reducing Gaussian-type noise while maintaining general image structure.

2. Median Filtering

Effective for removing salt-and-pepper noise while preserving edges.

3. Bilateral Filtering

Reduces noise while preserving important edges.

After preprocessing, robust feature detectors/descriptors such as **SIFT** or **ORB** can be used.

For matching, a distance-based method such as **BFMatcher** can be combined with a ratio test to remove ambiguous matches.

Solution Design

Improved pipeline:

Image 1 → Preprocessing → Feature Detection → Descriptor Extraction

Image 2 → Noise Reduction → Feature Detection → Descriptor Extraction

↓

Feature Matching → Match Filtering → Reliable Matches

A **ratio test** can reject ambiguous matches by comparing the nearest and second-nearest descriptor matches.

For geometrically consistent matching, **RANSAC** can further remove outlier matches by estimating a geometric transformation such as a homography.

Comparison

Noisy Image Processing	Preprocessed Image Processing
Many false keypoints	Fewer false keypoints
Descriptors are less stable	More stable descriptors
More incorrect matches	More correct matches
Low matching accuracy	Improved matching accuracy

Conclusion

Noise can significantly reduce feature-matching accuracy by creating false features and changing descriptor values. **Noise filtering followed by robust feature extraction, descriptor matching, ratio testing and RANSAC-based outlier removal** provides a more reliable matching system.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand